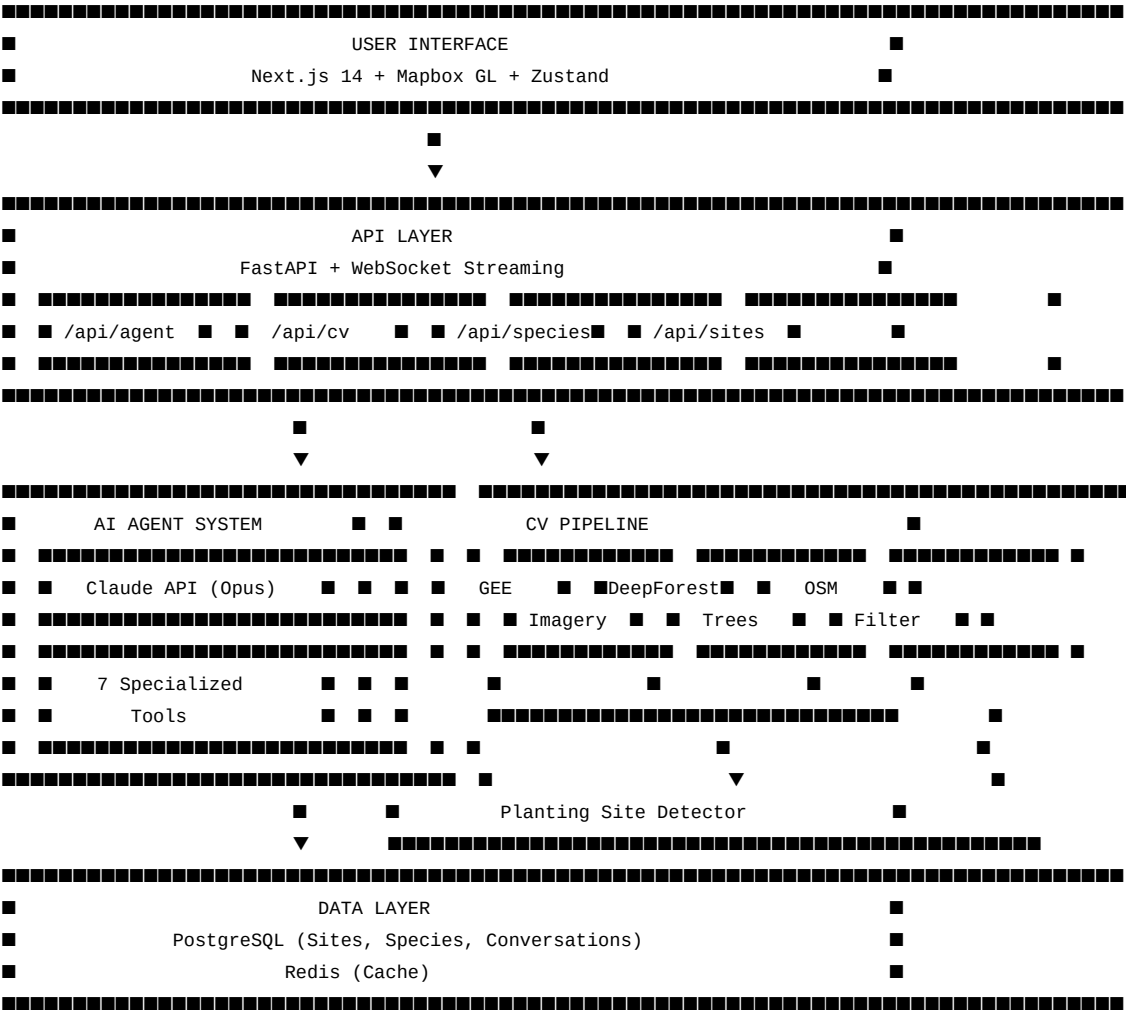


SmartCanopy Technical Documentation

Overview

SmartCanopy is an AI-powered urban tree planting system that identifies optimal planting locations using satellite imagery, computer vision, and conversational AI. The system analyzes vegetation health, terrain, and infrastructure to recommend suitable tree species with environmental benefit projections.

System Architecture

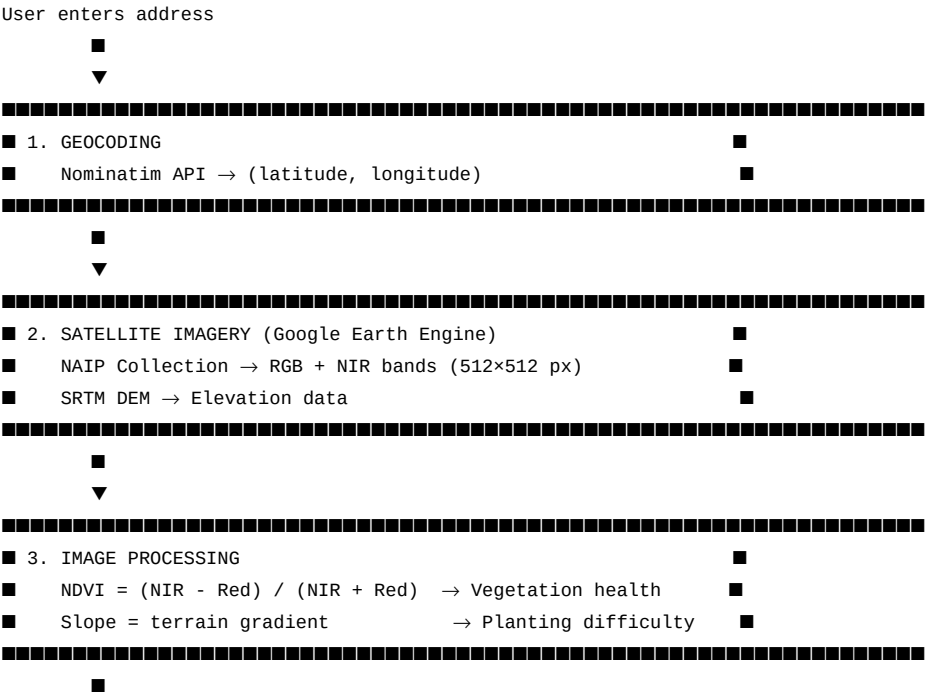


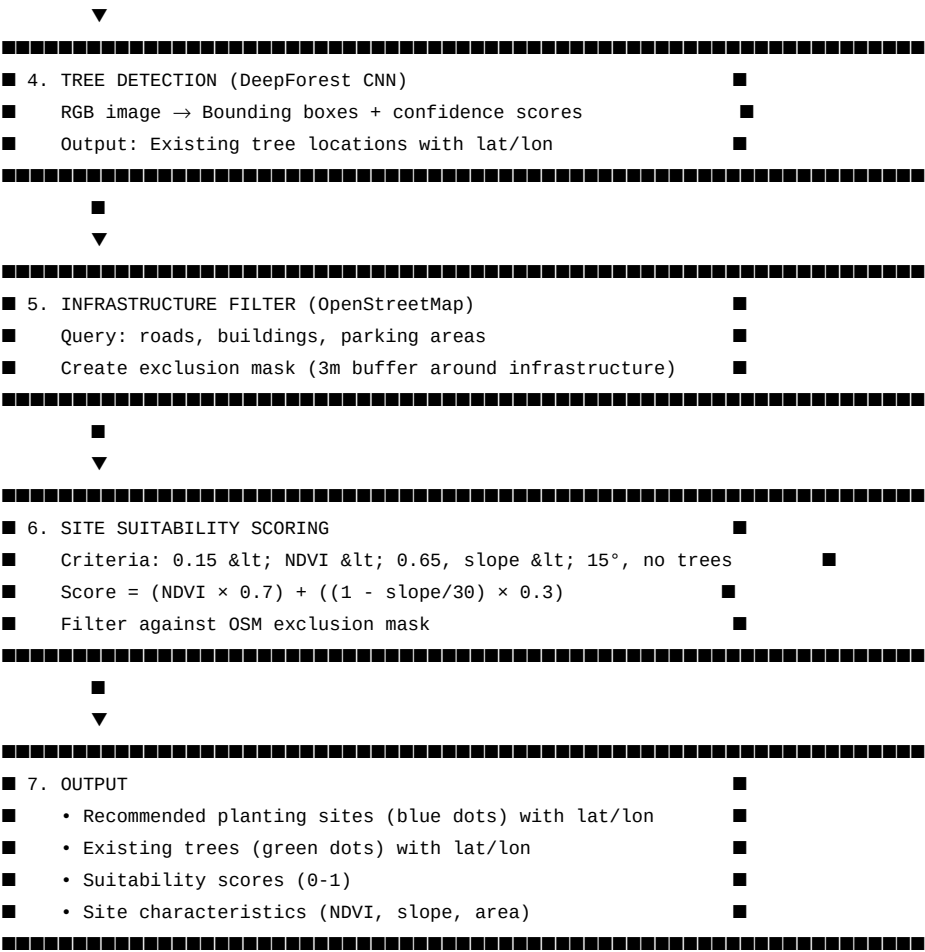
Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14, React 18	Web application framework
	Mapbox GL JS	Interactive mapping
	Zustand	State management
	Tailwind CSS	Styling
Backend	FastAPI	REST API + WebSocket
	Python 3.13	Runtime
	SQLAlchemy 2.0	ORM
AI	Claude Opus 4.5	Conversational agent
	DeepForest	Tree detection (CNN)
Data	PostgreSQL 16	Primary database
	Redis 7	Caching layer
External APIs	Google Earth Engine	Satellite imagery
	OpenStreetMap	Infrastructure data
	USDA APIs	Hardiness zones, species data

Data Flow

Complete Analysis Pipeline





Core Components

1. CV Pipeline

File	Class/Function	Purpose
`data_pipeline.py`	`TreeDataPipeline`	GEE imagery fetching, NDVI calculation
`download_imagery.py`	`ImageryDownloader`	Download RGB, NDVI, slope images
`test_deepforest.py`	`DeepForestDetector`	Tree detection using pre-trained CNN
`osm_filter.py`	`OSMFilter`	Infrastructure exclusion masks
`planting_site_detector.py`	`PlantingSiteDetector`	Orchestrates 7-step analysis

Key Function: pixel_to_latlon()

```
def pixel_to_latlon(px, py, center_lat, center_lon, buffer_m, img_width, img_height):
    """Convert pixel coordinates to geographic coordinates."""
    # Uses buffer radius and image dimensions to map pixels to lat/lon
```

2. AI Agent System

File	Class	Purpose
`agent/agent.py`	`SmartCanopyAgent`	Claude API integration, tool orchestration
`agent/config.py`	`Settings`	Environment configuration
`agent/tools/base_tool.py`	`BaseTool`	Abstract tool framework

7 Specialized Tools:

Tool	Purpose	Key Inputs
`species_recommender`	Match trees to sites	ZIP code, space, soil, sunlight
`pricing_calculator`	Cost estimates	Species, quantity, size, state
`environmental_calculator`	CO ₂ /stormwater benefits	Species, quantity, years
`hazard_checker`	Safety clearances	Species, lat/lon
`photo_analyzer`	Site photo analysis	Base64 image, question
`maintenance_guide`	Care instructions	Species, tree age, climate
`planting_instructions`	Step-by-step guides	Species, tree size, season

3. API Layer

Endpoint	Method	Purpose
`/api/cv/analyze`	POST	Run CV pipeline for address
`/api/agent/ws/{id}`	WebSocket	Real-time chat streaming
`/api/agent/chat`	POST	Non-streaming chat
`/api/species`	GET	List/search species
`/api/sites`	GET	List analyzed sites
`/api/health`	GET	Health check

WebSocket Message Types:

```
{ type: 'chunk', text: string }           // Streaming text
{ type: 'tool_use', tool_name: string }    // Tool execution started
{ type: 'tool_result', tool_name: string } // Tool complete
{ type: 'complete', full_response: string } // Response finished
{ type: 'error', message: string }        // Error occurred
```

4. Frontend

Component	File	Purpose
Map Container	`components/map/MapContainer.tsx`	Mapbox integration
Street Layer	`components/map/StreetLayer.tsx`	Trees + sites visualization
Chat Panel	`components/chat/ChatPanel.tsx`	AI conversation interface
Analysis Store	`stores/analysisStore.ts`	CV results state
Chat Store	`stores/chatStore.ts`	WebSocket + messages

Database Schema

Core Tables

```
-- Tree species catalog (469 species)
plant_species (
  species_id VARCHAR(10) PRIMARY KEY, -- e.g., "ACPL"
  common_name, scientific_name,
  mature_height_ft_min/max, mature_spread_ft_min/max,
  hardiness_zone_min/max,
  co2_sequestration_kg_year,
  stormwater_interception_gal_year,
  price_sapling, price_6ft, price_8ft, price_10ft
)

-- CV-analyzed planting locations
planting_sites (
  site_id UUID PRIMARY KEY,
  analysis_id UUID,
  address, latitude, longitude,
  avg_ndvi, avg_slope, area_pixels,
  suitability_score,
  rgb_image_path, ndvi_image_path, slope_image_path
)

-- Chat sessions
conversations (
  conversation_id UUID PRIMARY KEY,
  address, site_ids JSONB,
  messages JSONB,
  current_phase VARCHAR -- exploration|recommendation|planning|completed
)
```

Key Parameters & Thresholds

Component	Parameter	Value	Purpose
NDVI	Min threshold	0.15	Filter bare soil/pavement
NDVI	Max threshold	0.65	Exclude dense forest
Slope	Max threshold	15°	Ensure plantable terrain
DeepForest	Confidence	0.3	Tree detection threshold
OSM	Buffer	3 meters	Infrastructure clearance
Suitability	NDVI weight	70%	Scoring formula
Suitability	Slope weight	30%	Scoring formula
Buffer	Default radius	100m	Analysis area

External Service Integrations

Google Earth Engine

- Project ID: urban-tree-ai
- Collections: USDA/NAIP/DOQQ (imagery), USGS/SRTMGL1_003 (elevation)
- Auth: earthengine authenticate

DeepForest

- Pre-trained CNN for tree crown detection
- Automatic model download via `model.use_release()`
- Returns bounding boxes with confidence scores

OpenStreetMap (via osmnx)

- Queries: `highway=`, `building=`, `amenity=parking`
- Coordinate systems: WGS84 ↔ Web Mercator

Anthropic Claude API

- Model: `claude-opus-4-5-20251101`
- Features: Tool use, streaming, vision
- Max tokens: 4096

Running the System

Prerequisites

```
source venv/bin/activate
earthengine authenticate
docker-compose up -d postgres redis
```

Start Services

```
# Backend API (port 8000)
PYTHONPATH=/path/to/urban-tree-ai uvicorn api.main:app --reload

# Frontend (port 3000)
cd frontend && npm run dev
```

Test Endpoints

```
# Health check
curl http://localhost:8000/api/health

# CV analysis
curl -X POST http://localhost:8000/api/cv/analyze \
  -H "Content-Type: application/json" \
  -d '{"address": "123 Main St, San Jose, CA"}'
```

File Structure

```
urban-tree-ai/
├── planting_site_detector.py # Main CV orchestrator
├── data_pipeline.py          # GEE integration
├── download_imagery.py       # Image downloads
├── test_deepforest.py        # Tree detection
├── osm_filter.py             # Infrastructure filtering
├──
├── agent/
│   ├── agent.py              # Claude integration
│   ├── config.py             # Settings
│   ├── tools/                # 7 specialized tools
│   ├── services/             # Database, cache
│   └── prompts/              # System prompt
│   └──
├── api/
│   ├── main.py               # FastAPI app
│   └── routes/               # Endpoints
│   └──
├── frontend/
│   ├── src/
│   │   ├── app/              # Next.js pages
│   │   ├── components/       # React components
│   │   ├── stores/           # Zustand state
│   │   └── lib/              # API clients
```

```
■ ■■■ package.json
■
■■■ database/
■ ■■■ schema.sql          # PostgreSQL schema
■
■■■ docker-compose.yml    # Infrastructure
```

Visualization

Map Layers

Layer	Color	Data Source
Existing Trees	Green (#22c55e)	DeepForest detection
Recommended Sites	Blue (#3b82f6)	Suitability scoring
Infrastructure	Red overlay	OSM exclusion mask

Site Popup Information

- Priority score (1-10)
- Vegetation category (sparse/moderate/dense)
- Terrain category (flat/gentle/moderate/steep)
- Area (sq ft)
- Infrastructure warnings